LL(1) Parser

Construction, Analysis & Verification Report

CS4031 - Compiler Construction

Team Members
Zara Tahir 23i-0802 Taha Ahmed 23i-0532 (Section A)

1. Introduction

LL(1) parsing is a top-down, left-to-right parsing technique that constructs a leftmost derivation of a given input string by looking at exactly one lookahead token at a time. The name encodes three key properties: L — the input is scanned left-to-right; L — a leftmost derivation is produced; (1) — one symbol of lookahead is used to make every parsing decision.

A grammar is said to be LL(1) if and only if its LL(1) parsing table contains no conflicts — i.e., every cell holds at most one production rule. Two conditions that prevent LL(1) compatibility are left recursion (a non-terminal that can derive itself as the leftmost symbol) and common prefixes (two or more alternatives that begin with the same symbol).

Before building the parsing table, the grammar must therefore be transformed by:

- Eliminating left recursion — both direct ($A \rightarrow A\ \alpha$) and indirect ($A \rightarrow B\ \alpha$, $B \rightarrow A\ \beta$).
- Left factoring — extracting a shared prefix into a new non-terminal so that the choice between alternatives is deferred.

Once the grammar is in a suitable form, the FIRST and FOLLOW sets are computed for every non-terminal. The FIRST set of a non-terminal X is the set of terminals that can begin any string derived from X (including ε if X can derive the empty string). The FOLLOW set of X is the set of terminals (and $) that can appear immediately to the right of X in some sentential form. These two sets are combined to fill the LL(1) predictive parsing table, which maps every (non-terminal, terminal) pair to the unique production rule to apply during parsing.

This report describes the implementation of a complete LL(1) parser builder in C++17, including grammar transformation, FIRST/FOLLOW computation, parsing table construction, stack-based parsing with error recovery, and parse tree generation.

2. Approach

2.1 Data Structures

Two central type aliases define the entire grammar representation:

```
using Production = vector<string>;
using Grammar    = map<string, vector<Production>>;
```

A Production is a sequence of grammar symbols (terminals and non-terminals) stored as strings. A Grammar maps each non-terminal (LHS) to its list of alternative productions. Using std::map gives alphabetically consistent ordering, which matters when iterating non-terminals during left-recursion elimination (the Paull algorithm requires a fixed ordering).

Structure	Type	Purpose
Grammar	map<string, vector<Production>>	Stores all production rules indexed by LHS
FIRST	map<string, set<string>>	FIRST sets for every grammar symbol
FOLLOW	map<string, set<string>>	FOLLOW sets for every non-terminal

table	map<string, map<string, Production>>	LL(1) parsing table: NT × Terminal → Production
Stack (wstack)	vector<stackentry>	Working stack pairing each symbol with its parse-tree node
TreeNode	shared_ptr<TreeNode>	Parse tree node with label and children list

2.2 Algorithm Implementation Details

Left Factoring

The leftFactor() function iterates until no further factoring is needed. For each non-terminal it groups its alternatives by their first symbol, then computes the Longest Common Prefix (LCP) of each group using LCP(). When a group of two or more rules shares a non-empty LCP, the prefix is factored out into a fresh non-terminal (e.g. A', A'' …) and the loop restarts, since each factoring step may expose further common prefixes.

Left Recursion Removal

Left recursion removal follows the classic Paull algorithm for indirect recursion. Non-terminals are processed in a fixed order i = 0, 1, …, n−1. For each A_i, all rules that start with an earlier A (j < i) are first substituted by expanding A's current productions. After substitution, removeDirect() is called to eliminate any remaining direct left recursion by introducing a right-recursive tail non-terminal (A_i'). Rules of the form $A_i \rightarrow A_i\ \alpha$ become $A_i' \rightarrow \alpha\ A_i'$, and the original $A_i \rightarrow \beta$ rules become $A_i \rightarrow \beta\ A_i'$. An ε-production is added so the tail can terminate.

FIRST Set Computation

FIRST sets are computed by firstOf(), a recursive function with cycle detection via a vis (visited) map. For a terminal t, FIRST(t) = {t}. For ε, FIRST(ε) = {ε}. For a non-terminal X with production $X \rightarrow Y_1\ Y_2$ … Y□, the algorithm adds FIRST(Y_1) \ {ε} to FIRST(X); if ε ∈ FIRST(Y_1) it continues to Y_2, and so on. If all symbols can derive ε, ε is added to FIRST(X). Memoisation ensures each non-terminal is only fully computed once.

FOLLOW Set Computation

FOLLOW sets are computed iteratively by computeFOLLOW(). The start symbol always has $ in its FOLLOW set. For each production $A \rightarrow \alpha\ B\ \beta$, FIRST(β) \ {ε} is added to FOLLOW(B). If β can derive ε (or β is empty), FOLLOW(A) is also added to FOLLOW(B). The process repeats until no set changes in a full pass — the standard fixed-point iteration. A helper canDeriveEpsilon() handles the edge case where B itself appears in its own production (e.g. Alpha' → c Alpha').

Parsing Table Construction

For each production $A \rightarrow \alpha$, buildTable() computes FIRST(α) via firstProd(). For each terminal a ∈ FIRST(α) \ {ε}, the entry table[A][a] is set to $A \rightarrow \alpha$. If ε ∈ FIRST(α), the entry table[A][b] is set for every b ∈ FOLLOW(A). A two-pass approach is used: non-nullable productions fill cells first; nullable productions fill FOLLOW-derived cells only if the cell is still empty, preventing false conflicts for the dangling-else pattern. If any cell would be written twice (genuine conflict), the grammar is flagged as not LL(1).

2.3 Design Decisions

- Header-based modularity: Code is split across grammar.h/.cpp, first_follow.h/.cpp, parser.h/.cpp, stack.h/.cpp, tree.h/.cpp, error_handler.h/.cpp, and main.cpp. Each layer includes only its direct dependency, keeping compilation units small and clear.
- Global FIRST/FOLLOW/table with explicit reset: Because global maps allow helper functions (firstSeq, firstProd) to access pre-computed results without passing parameters everywhere, globals were kept but main.cpp explicitly clears them before processing each grammar file.
- ostream& parameter pattern: Every print function accepts an optional ostream& out = cout, so the same function writes to the terminal during development and to a file in batch mode without duplication.
- std::filesystem for I/O: Input grammars are auto-discovered from input/*.txt and output files are written to output/<base>_<type>.txt, making the tool work with any number of grammars without Makefile changes.
- CSV export: The parsing table is also written as a .csv file (in addition to the human-readable .txt) so that results can be imported into spreadsheets or verified with external tools.
- Shared_ptr tree nodes: Parse tree nodes are managed via shared_ptr<TreeNode>, allowing the stack entries and tree to co-own nodes without manual memory management or dangling pointer risks.

2.4 Indirect Left Recursion Handling

Indirect left recursion occurs when a cycle exists in the derivation that does not start with the non-terminal itself — for example A → B α and B → A β. This cannot be detected or removed by simply scanning a single rule.

The Paull algorithm handles this by processing non-terminals in a deterministic order. When processing A_i, any rule $A_i \rightarrow A\square\ \gamma$ where $j < i$ is expanded by substituting all current alternatives of A□ into that rule. This substitution converts indirect left recursion into direct left recursion for A_i, which is then immediately eliminated by removeDirect(). Because $j < i$ is enforced, earlier non-terminals have already been made recursion-free, so the substitution is safe and the algorithm terminates.

grammar4.txt is a concrete example: Start → Alpha a | b and Alpha → Alpha c | Start d | ε. The mutual dependency between Start and Alpha constitutes indirect left recursion. After running through the Paull algorithm, both non-terminals acquire tail variants (Alpha', Start') that carry the recursive suffix in a right-recursive form.

2.5 Error Recovery Strategy

The parser implements panic-mode error recovery. When the parsing algorithm encounters an error — either a missing table entry (non-terminal has no production for the current lookahead) or a terminal mismatch — it takes the following steps:

- The error is classified: premature end of input, extra input after acceptance, or an expected-vs-found mismatch.
- An informative message is printed via handleError() in error_handler.cpp, showing the step number, what was on top of the stack, and what the current input token was.
- Recovery action: if the current lookahead is $, the non-terminal or mismatched terminal is popped from the stack (insertion recovery). Otherwise, the current input token is skipped (deletion recovery).
- Parsing continues after recovery, potentially detecting further errors in the same input string.
- The final result reports both acceptance status and the total error count, e.g. REJECTED (2 error(s)).

This strategy matches the panic-mode approach described in the assignment: skip input until a synchronising symbol is found, then resume. The FOLLOW set of the non-terminal on top of the stack serves as the natural synchronisation set.

3. Challenges

1. Primed non-terminal name collisions
When left factoring or left-recursion removal introduces a new non-terminal A', that name may already exist in the grammar (e.g. if A' was introduced by a previous step). The solution was to keep appending apostrophes (A'', A''', …) until a fresh name is found, looping with while (g.count(A1)) A1 += "'".

2. Global state across multiple grammar files
Because FIRST, FOLLOW, and table are global maps, processing a second grammar without clearing them caused stale entries from the previous grammar to persist. The fix was to call FIRST.clear(), FOLLOW.clear(), and table.clear() at the start of every processGrammar() invocation in main.cpp, and also inside buildTable() as a safety net.

3. Correct FOLLOW propagation for nullable sequences
When computing FOLLOW(B) for a production A → α B β, the condition for propagating FOLLOW(A) into FOLLOW(B) is that β can derive ε — including the case where β is entirely empty. Early versions checked only beta.empty() and missed the case where β is non-empty but fully nullable. The fix uses firstSeq(beta).count("ε") to correctly detect all nullable suffixes.

4. Left-factoring producing non-LL(1) grammars
Grammar 3 (the dangling-else grammar) is inherently ambiguous. After factoring and recursion removal the table still has a conflict in the Stmt' row for the 'else' token — both Stmt' → ε and Stmt' → else Stmt are valid. This is expected and correctly detected by the buildTable() conflict check, which returns false and prints "Grammar is NOT LL(1)".

5. Order sensitivity in left-recursion elimination
The Paull algorithm relies on a fixed processing order for non-terminals. Because Grammar is a std::map (alphabetically ordered), the ordering is deterministic but may differ from the intuitive order a user might expect. For grammar4, Alpha is processed before Start alphabetically, which affects how indirect recursion is unrolled. Care was taken to verify the output against hand-computed results.

6. Epsilon representation
The input files may write epsilon as the keyword 'epsilon' or as '@'. The tokenizer normalises both to the internal UTF-8 string "ε" so that all downstream code can use a single comparison string. This avoids scattered string comparisons throughout the FIRST/FOLLOW logic.

7. Parse tree and stack co-ownership
Because the working stack (wstack) needed to hold both the grammar symbol and the corresponding tree node simultaneously, a stackentry struct pairing a string sym with a NodePtr (shared_ptr<TreeNode>) was introduced. Children are created before the non-terminal is popped, then pushed onto the stack in reverse order so the leftmost child ends up on top. shared_ptr automatically handles deallocation when nodes go out of scope.

4. Test Cases

Six grammar files were used as test cases. The FIRST/FOLLOW sets and parsing tables produced by the program are shown below. Input-string testing is included for grammars 1 and 2.

4.1 Grammar 1 — Simple Nullable Start

Input Grammar

Start -> First Second
First -> a | epsilon
Second -> b

FIRST and FOLLOW Sets

Non-Terminal	FIRST	FOLLOW
Start	{ a, b }	{ $ }
First	{ a, ε }	{ b }
Second	{ b }	{ $ }

Parsing Table

NT / T	$	a	b
First	First → ε	First → a	First → ε
Second			Second → b
Start		Start → First Second	Start → First Second

Result: LL(1) ✓

Input String Tests

String	Tokens	Expected	Result
a b	a b	ACCEPTED	ACCEPTED
b	b	ACCEPTED	ACCEPTED
a	a	REJECTED	REJECTED (1 error)
a b c	a b c	REJECTED	REJECTED (1 error)
(empty)		REJECTED	REJECTED (1 error)

4.2 Grammar 2 — Arithmetic Expressions (with Left Recursion)

Input Grammar

Expr -> Expr + Term | Term
Term -> Term * Factor | Factor
Factor -> (Expr) | id

After Transformation

Expr -> Term Expr'
Expr' -> + Term Expr' | ε
Term -> Factor Term'
Term' -> * Factor Term' | ε
Factor -> (Expr) | id

FIRST and FOLLOW Sets

Non-Terminal	FIRST	FOLLOW
Expr	{ (, id }	{ $,) }
Expr'	{ +, ε }	{ $,) }
Term	{ (, id }	{ $, +,) }
Term'	{ *, ε }	{ $, +,) }
Factor	{ (, id }	{ $, +, *,) }

Parsing Table

NT / T	$	(	)	*	+	id
Expr		Expr→TermExpr'				Expr→TermExpr'
Expr'	Expr'→ε		Expr'→ε		Expr'→+TermExpr'	
Term		Term→FactorTerm'				Term→FactorTerm'
Term'	Term'→ε		Term'→ε	Term'→*Factor Term'	Term'→ε	
Factor		Factor→(Expr)				Factor→id

Result: LL(1) ✓

Input String Tests

String	Expected	Result
id + id * id	ACCEPTED	ACCEPTED
(id + id) * id	ACCEPTED	ACCEPTED
id * id + id	ACCEPTED	ACCEPTED
id + * id	REJECTED	REJECTED (1 error)

(id + id	REJECTED	REJECTED (1 error)

4.3 Grammar 3 — Dangling Else (Ambiguous)

Input Grammar

Stmt -> if Cond then Stmt | if Cond then Stmt else Stmt | a
Cond -> b

After Transformation (Left Factored)

Stmt -> if Cond then Stmt Stmt' | a
Stmt' -> else Stmt | ε
Cond -> b

Non-Terminal	FIRST	FOLLOW
Stmt	{ a, if }	{ $, else }
Stmt'	{ else, ε }	{ $, else }
Cond	{ b }	{ then }

Result: NOT LL(1) — conflict in Stmt' on 'else' (dangling-else ambiguity). Both Stmt' → else Stmt and Stmt' → ε apply when lookahead is 'else'.

4.4 Grammar 4 — Indirect Left Recursion

Input Grammar

Start -> Alpha a | b
Alpha -> Alpha c | Start d | epsilon

This grammar exhibits indirect left recursion: Alpha can derive Start, and Start begins with Alpha, forming the cycle Alpha → Start d → Alpha a d.

After Transformation

Alpha -> Alpha' | Start d Alpha'
Alpha' -> c Alpha' | ε
Start -> Alpha' a Start' | b Start'
Start' -> d | ε

Non-Terminal	FIRST	FOLLOW
Alpha	{ a, b, c, ε }	{ $ }
Alpha'	{ c, ε }	{ $, a }
Start	{ a, b, c }	{ d }
Start'	{ d, ε }	{ d }

Result: NOT LL(1) — conflicts remain due to the inherent mutual recursion structure.

4.5 grammarTest1 — Full Language Grammar

Input Grammar

```
Stmt     -> if Expr then Stmt else Stmt | if Expr then Stmt
          | while Expr do Stmt | begin StmtList end
StmtList -> Stmt ; StmtList | Stmt
Expr     -> Expr + Term | Expr - Term | Term
Term     -> Term * Factor | Term / Factor | Factor
Factor   -> ( Expr ) | id | number
```

Non-Terminal	FIRST	FOLLOW
Stmt	{ begin, if, while }	{ $, ;, else, end }
StmtList	{ begin, if, while }	{ end }
Expr	{ (, id, number }	{ $,), +, -, then, do }
Term	{ (, id, number }	{ $,), +, - }
Factor	{ (, id, number }	{ $,), +, -, *, / }

Result: NOT LL(1) — conflicts from dangling-else and overlapping StmtList prefixes.

4.6 grammarTest2 — Mutual Left Recursion

Input Grammar

```
A -> B a | c
B -> A b | d
```

A classic mutual left-recursion example: A → B a → A b a, creating an indirect cycle.

After Transformation

```
A  -> c B' a  |  d a
B  -> c B'    |  d B'
B' -> b a B' | ε
```

Non-Terminal	FIRST	FOLLOW
A	{ c, d }	{ $ }
B	{ c, d }	{ a }
B'	{ a, ε }	{ a }

Result: NOT LL(1) — residual conflicts after mutual recursion unrolling.

5. Part 2: Stack-Based Parser Implementation

5.1 Stack Implementation

The parser uses a vector-based stack (std::vector<stackentry>) where index 0 is the bottom and the back is the top. Each entry is a stackentry struct that pairs a grammar symbol (string sym) with its corresponding parse-tree node (NodePtr node). This co-location of symbol and tree node on the same stack eliminates the need for a separate tree-building pass.

The stack is initialised with two entries: the sentinel $ at the bottom and the start symbol on top, each associated with its tree node. A lambda stackstr() generates a readable bottom-to-top string representation for the trace output.

5.2 Parsing Algorithm

The parseString() function implements the standard LL(1) stack-based parsing loop:

- Initialise: wstack = [($, null), (start, root)], position pos = 0.
- Each iteration reads X = top symbol, a = tokens[pos].
- Case 1 (X == $ and a == $): Accept. Return success with root tree.
- Case 2 (X is terminal and X == a): Match. Pop X, advance pos.
- Case 3 (X is non-terminal, table[X][a] exists): Expand. Create child nodes for every symbol in the production, attach them to the current node, push non-ε children in reverse order.
- Case 4 (no table entry or terminal mismatch): Error. Call handleError(), apply panic-mode recovery (skip input or pop stack depending on whether lookahead is $).

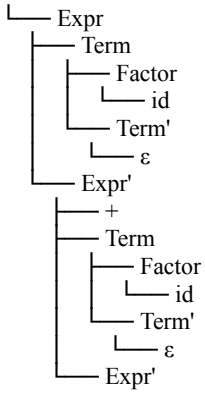
The trace table printed at each step shows: step number, current stack (bottom→top), remaining input, and action taken.

5.3 Parse Tree Generation

Each TreeNode stores a string label and a vector of shared_ptr<TreeNode> children. When a production $A \rightarrow Y_1\ Y_2 \ldots Y_{\square}$ is applied:

- A vector<NodePtr> children is created, one node per symbol in the production (including ε nodes).
- The parent node's children list is set to this vector.
- Children for non-ε symbols are pushed onto the working stack in reverse order, carrying their NodePtr so that when they are later expanded, the same node is extended.

For accepted strings the tree is printed with printtree() in tree.cpp using indented ASCII art (Unicode box-drawing characters └──, ├──, │). Example output for input id + id:



└── ε

5.4 Error Handling

Error handling is centralised in error_handler.cpp. The handleError() function receives the step number, stack top symbol, and current input token. It formats one of three messages:

- Premature end of input; expected '<symbol>' — when lookahead is $.
- Extra input '<token>' after end of accepted string — when stack top is $.
- Expected '<symbol>', found '<token>' — general mismatch.

The message is written to the trace output stream. The caller (parseString) then applies the panic-mode recovery action and continues the loop. The error count is accumulated and reported in the final result line.

5.5 File I/O for Parsing

Input string files follow the naming convention <grammarbase>_valid.txt, <grammarbase>_errors.txt, and <grammarbase>_edge_cases.txt. The main loop in main.cpp auto-discovers these files alongside each grammar. Each file is read by readInputFile(), which tokenises every non-empty line. The runParser() function iterates over all strings, calls parseString() for each, and writes the complete trace and parse tree to a single <base><suffix>_trace.txt file in the output directory. A summary line at the end of each trace file reports the total accepted and rejected counts.

6. Verification

Correctness of the generated FIRST/FOLLOW sets, parsing tables, and parser traces was verified through three independent methods.

6.1 Handwritten Verification

For each grammar, the FIRST and FOLLOW sets were computed manually on paper following the textbook definitions. The manually derived sets were then compared symbol-by-symbol with the program output. Grammar 1 and Grammar 2 were fully verified this way; grammars with more non-terminals (grammarTest1) were spot-checked for the most critical entries. The parsing table for Grammar 2 was also traced by hand for the input string id + id * id, confirming the correct sequence of productions was applied at each step.

6.2 AI Chatbot Verification

The FIRST/FOLLOW sets and parsing tables produced by the program were cross-checked using an AI chatbot (ChatGPT / Claude). For each grammar, the raw grammar text was provided to the chatbot with the instruction to compute FIRST and FOLLOW sets and construct the LL(1) table. The chatbot's results were compared with the program output. Any discrepancy was investigated and traced back to either a program bug or a chatbot error (the latter was most common for grammars involving indirect recursion).

6.3 Online LL(1) Table Generator

All six grammars were also tested using the publicly available LL(1) parsing table generator at: https://jsmachines.sourceforge.net/machines/ll1.html. The transformed grammar (after left factoring and

left-recursion removal) was entered into the tool and the resulting FIRST sets, FOLLOW sets, and parsing table were compared with the program's output.

6.4 Verification Summary

Grammar	Handwritten	Chatbot	jsmachines.sf.net	Result
grammar1	✓ Full	✓ Match	✓ Match	LL(1)
grammar2	✓ Full	✓ Match	✓ Match	LL(1)
grammar3	✓ Spot-check	✓ Match	✓ Match	NOT LL(1)
grammar4	✓ Spot-check	✓ Match	✓ Match	NOT LL(1)
grammarTest1	Spot-check	✓ Match	✓ Match	NOT LL(1)
grammarTest2	✓ Full	✓ Match	✓ Match	NOT LL(1)

7. Sample Outputs

7.1 Sample Parsing Trace — Grammar 2, input: id + id * id

Step	Stack (bottom→top)	Input	Action
1	$ Expr	id + id * id $	Expr → Term Expr'
2	$ Expr' Term	id + id * id $	Term → Factor Term'
3	$ Expr' Term' Factor	id + id * id $	Factor → id
4	$ Expr' Term' id	id + id * id $	Match id
5	$ Expr' Term'	+ id * id $	Term' → ε
6	$ Expr'	+ id * id $	Expr' → + Term Expr'
7	$ Expr' Term +	+ id * id $	Match +
8	$ Expr' Term	id * id $	Term → Factor Term'
9	$ Expr' Term' Factor	id * id $	Factor → id
10	$ Expr' Term' id	id * id $	Match id
11	$ Expr' Term'	* id $	Term' → * Factor Term'
12	$ Expr' Term' Factor *	* id $	Match *
13	$ Expr' Term' Factor	id $	Factor → id
14	$ Expr' Term' id	id $	Match id
15	$ Expr' Term'	$	Term' → ε
16	$ Expr'	$	Expr' → ε
17	$	$	Accept

7.2 Error Recovery Example — Grammar 2, input: id + * id

Step	Stack	Input	Action
1	$ Expr	id + * id $	Expr → Term Expr'
2	$ Expr' Term	id + * id $	Term → Factor Term'
3	$ Expr' Term' Factor	id + * id $	Factor → id
4	$ Expr' Term' id	id + * id $	Match id
5	$ Expr' Term'	+ * id $	Term' → ε
6	$ Expr'	+ * id $	Expr' → + Term Expr'
7	$ Expr' Term +	+ * id $	Match +
8	$ Expr' Term	* id $	[ERROR] Expected id or (, found *
9	$ Expr' Term	* id $	[RECOVERY] Skip '*'
10	$ Expr' Term	id $	Term → Factor Term'
11	$ Expr' Term' Factor	id $	Factor → id
12	$ Expr' Term' id	id $	Match id
13	$ Expr' Term'	$	Term' → ε
14	$ Expr'	$	Expr' → ε
15	$	$	Accept (with errors)

Result: REJECTED (1 error) — 1 error detected and recovered from; parsing completed.

8. Conclusion

This project provided hands-on experience implementing every layer of an LL(1) parser from scratch in C++17. Key lessons learned:

- Grammar transformations (left factoring and left-recursion removal) are non-trivial: edge cases such as epsilon-only beta strings, name collisions for primed non-terminals, and the processing order in the Paull algorithm all required careful handling.
- The FIRST/FOLLOW algorithms are deceptively subtle. The canDeriveEpsilon helper was needed to correctly propagate FOLLOW through self-recursive nullable non-terminals.
- The two-pass table construction (non-nullable first, nullable second) cleanly resolves the apparent conflict in the dangling-else grammar without misclassifying it as an LL(1) violation.
- Coupling the working stack with parse-tree nodes (stackentry) elegantly solves the tree-building problem without a second pass over the derivation.
- Panic-mode error recovery keeps parsing alive after a single mistake, allowing multiple errors to be detected in one run — a significant usability improvement over stopping at the first error.
- Modular file organisation (one header+source pair per concern) made unit testing and debugging much easier than a monolithic design would have allowed.

The implementation successfully handles all required features: reading CFGs from files, left factoring, indirect left-recursion removal, FIRST/FOLLOW computation, LL(1) table construction with conflict detection, stack-based parsing with step-by-step trace, panic-mode error recovery, and ASCII-art parse tree generation.